

RNCP 37873 · NIVEAU 6 · CDA

ALVIO

AI-Powered Trading Platform

Candidat : Adam Ijjai

Titre visé : Concepteur Développeur d'Applications

Référentiel : RNCP 37873 · Niveau 6 (Bac+3/4)

Stack : NestJS · Next.js · React Native · Python · PostgreSQL

Date : Juin 2026

ALVIO — Dossier de Projet · Certification RNCP Niveau 6

Concepteur Développeur d'Applications — RNCP 37873

Candidat : Adam Ijjai

Titre visé : CDA — Concepteur Développeur d'Applications (RNCP 37873, Niveau 6)

Projet : ALVIO — AI-Powered Trading Platform

Date : Juin 2026

SECTION 1 — PRÉSENTATION DU PROJET

1.1 Identité du projet

Champ	Valeur
Nom	Alvio
Slogan	AI-Powered Trading Platform
Type	Application web fullstack + mobile + module IA
Stack principale	NestJS · Next.js · React Native · Python · PostgreSQL
Public cible	Traders particuliers, investisseurs débutants à avancés

1.2 Description du projet

Alvio est une plateforme de trading algorithmique propulsée par l'intelligence artificielle, conçue pour démocratiser l'accès aux stratégies de trading quantitatif. La plateforme analyse en temps réel les marchés crypto via l'API CoinGecko, détecte automatiquement des patterns techniques (chandeliers japonais, figures chartistes, patterns harmoniques, vagues Elliott) et génère des signaux de trading précis avec direction (BUY/SELL/HOLD), prix d'entrée, stop-loss, take-profit et score de confiance.

L'application intègre le modèle de langage Claude (Anthropic) pour analyser des documents PDF de stratégies de trading et les convertir en règles algorithmiques exploitables. Un module de formation intégré (LMS) permet aux utilisateurs d'apprendre les bases du trading à travers des cours structurés par niveau. La sécurité de l'application est de niveau bancaire, avec une authentification multi-facteurs complète (TOTP RFC 6238, WebAuthn FIDO2, PIN).

1.3 Problème résolu

Les plateformes de trading professionnel (Bloomberg Terminal, Reuters Eikon) sont inaccessibles au grand public en raison de leur coût (des milliers d'euros par mois) et de leur complexité. Les traders particuliers n'ont pas accès aux outils d'analyse technique avancée, ni

à l'IA pour les aider à déchiffrer les marchés. De plus, la courbe d'apprentissage du trading algorithmique est très raide.

1.4 Solution apportée

Alvio résout ce problème en offrant :

- **Analyse IA en temps réel** : 12 modules Python de détection de patterns + indicateurs techniques
- **Import de stratégies via Claude AI** : upload d'un PDF → extraction automatique des règles
- **Formation intégrée** : LMS avec cours, leçons, quiz et suivi de progression
- **Accessibilité** : interface web Next.js + application mobile React Native
- **Sécurité entreprise** : MFA complet (TOTP, WebAuthn FIDO2, GitHub OAuth, Magic Link)
- **Simulateur DCA** : test de stratégies d'investissement sans risque réel

SECTION 2 — STACK TECHNIQUE COMPLÈTE

2.1 Backend

Technologie	Version	Rôle dans le projet
NestJS	^10.0.0	Framework backend principal — architecture modulaire, DI, guards, interceptors
TypeScript	~5.1.3	Typage statique intégral backend + frontend
Node.js	18+	Runtime JavaScript côté serveur
Prisma	^5.0.0	ORM — modèles de données, migrations, client typé
PostgreSQL	15 (Docker)	Base de données relationnelle principale
Redis	7 (Docker)	Sessions, tokens MFA, challenges WebAuthn, rate-limiting
JWT (@nestjs/jwt)	^10.1.3	Access tokens (15 min) + Refresh tokens (7 jours)
Passport.js	^10.0.3	Stratégies d'authentification (JWT, GitHub OAuth)
bcrypt	—	Hash mots de passe (12 rounds) et PIN
otplib	—	Génération/vérification TOTP RFC 6238
@simplewebauthn/server	—	WebAuthn FIDO2 (enrôlement + authentification biométrique)
qrcode	—	Génération QR codes TOTP
Nodemailer	—	Envoi d'emails SMTP (magic link, OTP, notifications)
Anthropic SDK	^0.100.1	Appel Claude claude-sonnet-4-6 pour analyse de stratégies
prom-client	—	Métriques Prometheus (latence, compteurs, gauges)
Axios	^1.4.0	Client HTTP

2.2 Frontend Web

Technologie	Version	Rôle dans le projet
Next.js	13.4.0	Framework React SSR/CSR — pages, routing, API routes proxy
React	18.2.0	UI composants, hooks, state management local
TypeScript	5.1.3	Typage statique complet
Tailwind CSS	^3.3.0	Styling utilitaire — responsive design
Zustand	^4.5.7	State management global (auth store in-memory)
Axios	^1.4.0	Client HTTP avec interceptors JWT + refresh automatique
lightweight-charts	^4.2.3	Graphiques chandeliers japonais haute performance (TradingView)
framer-motion	^12.34.1	Animations UI fluides
@simplewebauthn/browser	^10.0.0	API WebAuthn côté navigateur (biométrie)
next-themes	^0.4.6	Gestion thème clair/sombre

2.3 Mobile

Technologie	Version	Rôle dans le projet
React Native	—	Application mobile cross-platform (iOS + Android)
Expo	—	Toolchain React Native
TypeScript	—	Typage statique

2.4 Module IA Python

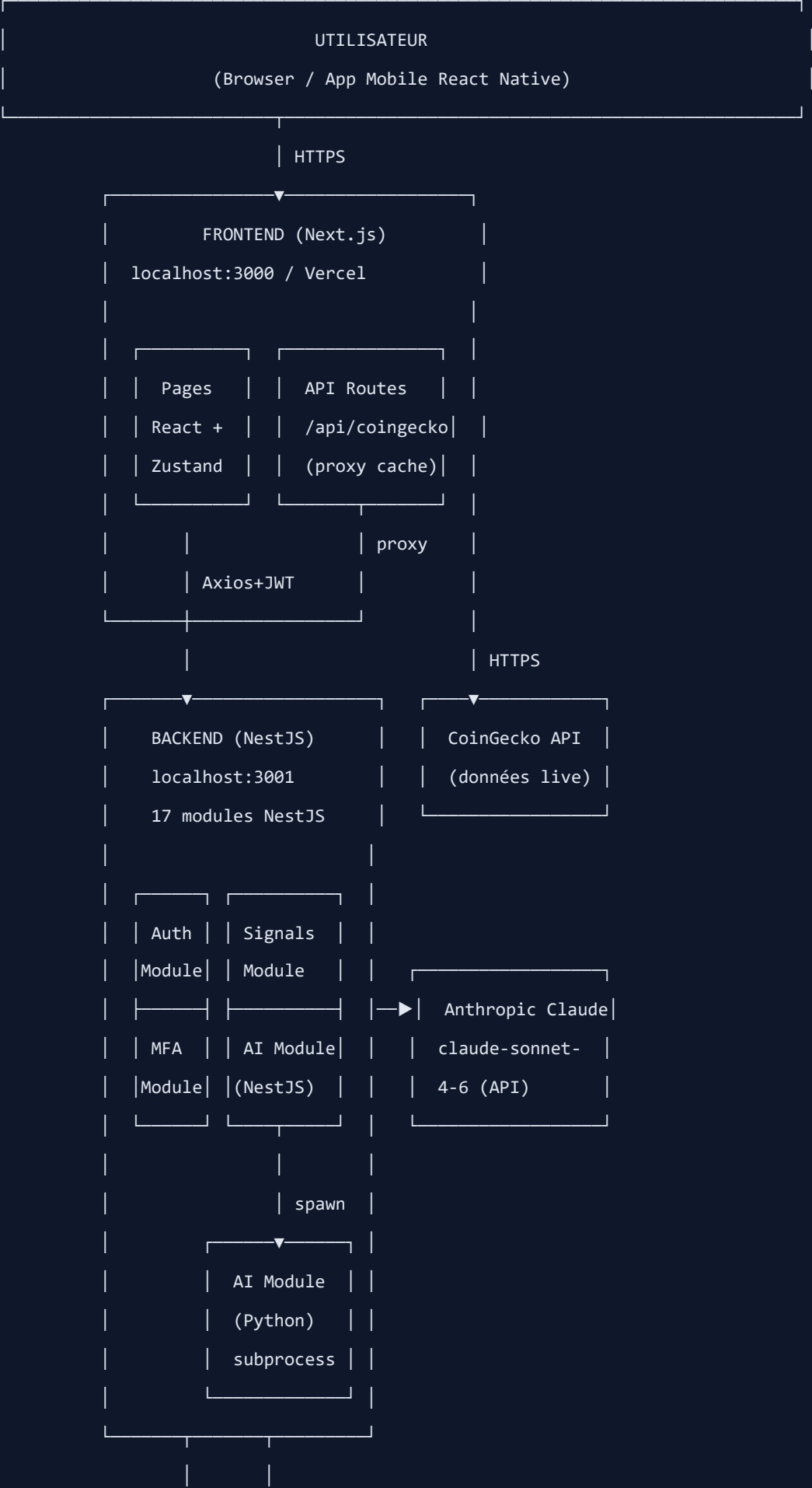
Technologie	Version	Rôle dans le projet
Python	3.10+	Runtime module IA
pandas	—	Manipulation DataFrames OHLCV
numpy	—	Calculs mathématiques (indicateurs techniques)
scikit-learn	—	Machine learning (scoring, classification patterns)
ta-lib / calculs manuels	—	Indicateurs techniques (RSI, MACD, Bollinger, ATR, Stochastic)
Anthropic Claude API	—	Analyse NLP de documents de stratégie (via backend)

2.5 Infrastructure & DevOps

Technologie	Version	Rôle dans le projet
Docker	29.0.1	Conteneurisation PostgreSQL, Redis, backend, frontend
Docker Compose	v3.8	Orchestration stack complète (9 services)
Prometheus	v2.51.0	Collecte métriques backend
Grafana	10.4.0	Dashboards monitoring
Elasticsearch	8.13.0	Moteur SIEM — logs de sécurité
Kibana	8.13.0	UI SIEM — visualisation logs
Logstash	8.13.0	Pipeline Syslog UDP → Elasticsearch

SECTION 3 — ARCHITECTURE DU PROJET

3.1 Architecture globale



PostgreSQL	Redis
(Prisma)	Sessions
localhost:	MFA keys
5432	6379

► Flux de données principaux

1. **Auth flow** : Browser → `POST /auth/login` → AuthService → bcrypt compare → Redis (issueTokens) → httpOnly cookie
2. **Signal generation** : Backend → AIService → `python signal_generator.py` → CandlestickDetector + Indicators → TradingSignal → Prisma → EmailService
3. **Strategy import** : Frontend upload PDF → `POST /strategies/import` → extractText → `client.messages.create(Claude)` → JSON StrategyRules → Prisma
4. **Markets live** : Browser → `GET /api/coingecko/markets` → Next.js API Route → serverCache (TTL 60s) → CoinGecko API → JSON coins
5. **Candlestick chart** : Browser → `GET /api/coingecko/ohlcv/[id]` → [t,o,h,l,c][] → lightweight-charts → canvas render

3.2 Structure des dossiers

brokeria/08_Code_Base/

```
|
|
| └─ backend-code/                                # API NestJS
|   | └─ src/
|   |   | └─ main.ts                             # Bootstrap NestJS (app, CORS, pipes, filters)
|   |   | └─ app.module.ts                       # Module racine – imports tous les modules
|   |   | └─ app.controller.ts                   # GET / (health check)
|   |   |
|   |   | └─ auth/                               # Authentification complète
|   |   |   | └─ auth.module.ts
|   |   |   | └─ auth.controller.ts              # 12 endpoints auth
|   |   |   | └─ auth.service.ts                # 520 lignes – logique auth complète
|   |   |   | └─ guards/
|   |   |   |   | └─ jwt-auth.guard.ts          # Guard JWT (@UseGuards)
|   |   |   |   | └─ strategies/
|   |   |   |   |   | └─ jwt.strategy.ts        # Passport JWT strategy
|   |   |   |   |   | └─ github.strategy.ts    # Passport GitHub OAuth strategy
|   |   |   |   | └─ dto/                      # Data Transfer Objects validés
|   |   |   |   |   | └─ register.dto.ts
|   |   |   |   |   | └─ login.dto.ts
|   |   |   |   |   | └─ magic-link.dto.ts
|   |   |   |   |   | └─ verify-2fa.dto.ts
|   |   |   |   |   | └─ setup-pin.dto.ts
|   |   |   |   |   | └─ verify-pin.dto.ts
|   |   |   |   |   | └─ set-password.dto.ts
|   |   |   |
|   |   |   | └─ mfa/                          # Multi-Factor Authentication
|   |   |   |   | └─ totp/                      # TOTP RFC 6238
|   |   |   |   |   | └─ totp.module.ts
|   |   |   |   |   | └─ totp.controller.ts     # 5 endpoints TOTP
|   |   |   |   |   | └─ totp.service.ts        # 195 lignes – enrôlement + vérification
|   |   |   |   |   | └─ totp.service.spec.ts   # Tests unitaires
|   |   |   |   | └─ webauthn/                  # FIDO2 / WebAuthn
|   |   |   |   |   | └─ webauthn.module.ts
|   |   |   |   |   | └─ webauthn.controller.ts
|   |   |   |   |   | └─ webauthn.service.ts    # 295 lignes – biométrie FIDO2
|   |   |   |   |   | └─ webauthn.service.spec.ts
|   |   |   |
|   |   |   | └─ ai/                          # Module IA NestJS
```

```
| | | └─ ai.module.ts
| | | └─ ai.controller.ts
| | | └─ ai.service.ts          # 280 lignes – Claude + Python subprocesses
| | | └─ interfaces/
| | |     └─ strategy-rules.interface.ts
| | |
| | | └─ signals/              # Signaux de trading
| | |     └─ signals.module.ts
| | |     └─ signals.controller.ts
| | |     └─ signals.service.ts  # 106 lignes – CRUD + email notification
| | |     └─ dto/create-signal.dto.ts
| | |
| | | └─ strategies/          # Stratégies de trading
| | |     └─ strategies.module.ts
| | |     └─ strategies.controller.ts
| | |     └─ strategies.service.ts  # Import PDF → Claude AI
| | |     └─ dto/import-strategy.dto.ts
| | |
| | | └─ formation/           # LMS – module formation
| | |     └─ formation.module.ts
| | |     └─ formation.controller.ts
| | |     └─ formation.service.ts  # 172 lignes – cours + progression
| | |     └─ dto/create-progress.dto.ts
| | |
| | | └─ simulator/           # Simulateur DCA
| | |     └─ simulator.module.ts
| | |     └─ simulator.controller.ts
| | |     └─ simulator.service.ts
| | |
| | | └─ database/            # Prisma
| | |     └─ prisma.module.ts
| | |     └─ prisma.service.ts    # PrismaClient provider injectable
| | |
| | | └─ redis/               # Redis (@Global)
| | |     └─ redis.module.ts
| | |     └─ redis.service.ts     # get/set/del wrappers avec TTL
| | |
| | | └─ email/               # Service email
| | |     └─ email.module.ts
| | |     └─ email.service.ts     # 314 lignes – 3 templates HTML
```

```

| | |
| | └─ logging/                # Audit & logs (@Global)
| |   └─ logging.module.ts
| |     └─ logging.service.ts  # 238 lignes – console + DB + syslog UDP
| |
| | └─ metrics/                # Prometheus (@Global)
| |   └─ metrics.module.ts
| |     └─ metrics.service.ts  # prom-client – 6 métriques
| |
| | └─ users/                  # Module utilisateurs
| | └─ reports/                # Module rapports
| | └─ payments/               # Module paiements
| |   └─ common/
| |     └─ filters/
| |       └─ http-exception.filter.ts
| |       └─ interceptors/
| |         └─ transform.interceptor.ts
| |
| | └─ prisma/
| |   └─ schema.prisma          # 216 lignes – 8 modèles
| |   └─ seed-formation.ts      # Seed données formation
| |
| | └─ .env                     # Variables d'environnement
| | └─ package.json
| |   └─ tsconfig.json
| |
| └─ frontend-web/              # Application Next.js
|   └─ pages/
|     └─ index.tsx              # Landing page
|     └─ login.tsx / register.tsx
|     └─ dashboard/index.tsx    # Dashboard KPI principal
|     └─ signals/
|       └─ index.tsx            # Liste signaux + filtres
|       └─ [id].tsx             # Détail signal
|     └─ markets/
|       └─ index.tsx            # Liste marchés crypto (CoinGecko)
|       └─ [id].tsx             # Détail marché + chandelier
|     └─ strategies/
|       └─ index.tsx / new.tsx
|       └─ import.tsx          # Import PDF → Claude AI

```

```
| | └─ simulator/index.tsx          # Simulateur DCA
| | └─ formation/
| |   └─ index.tsx                  # Liste cours
| |   └─ [courseId]/index.tsx       # Détail cours
| |   └─ [courseId]/[lessonId].tsx # Lecteur leçon
| | └─ reports/index.tsx
| | └─ pricing/index.tsx
| | └─ profile/
| |   └─ index.tsx                  # Profil utilisateur
| |   └─ security.tsx              # Sécurité MFA
| | └─ auth/                        # Toutes les pages auth MFA
| |   └─ 2fa.tsx / pin.tsx
| |   └─ setup-pin.tsx
| |   └─ totp-setup.tsx / totp-verify.tsx
| |   └─ webauthn-setup.tsx
| |   └─ magic.tsx                  # Handler magic link
| |   └─ github-callback.tsx
| |   └─ create-password.tsx
| |   └─ locked.tsx                # Compte verrouillé
| | └─ api/
| |   └─ coingecko/                # API Routes proxy (cache serveur)
| |     └─ markets.ts
| |     └─ coins/[id].ts
| |     └─ ohlc/[id].ts
| |
| | └─ components/
| |   └─ layout/
| |     └─ AppLayout.tsx           # Layout principal (sidebar + header)
| |     └─ Layout.tsx
| |     └─ PageTransition.tsx
| |   └─ common/
| |     └─ SignalCard.tsx          # Carte signal trading
| |     └─ StatCard.tsx            # Carte KPI
| |     └─ DashboardFuturiste.tsx # Dashboard principal
| |     └─ PatternChart.tsx        # Chart patterns
| |     └─ ProjectionLongTerme.tsx # Projection portefeuille
| |     └─ SignalIcon.tsx
| |     └─ AvatarIA.tsx
| |     └─ ThemeToggleButton.tsx
| |
```


3.3 Base de données — Modèles Prisma

▸ Modèles et relations

```
User (1) ————— (n) Signal
      (1) ————— (n) Strategy
      (1) ————— (n) Report
      (1) ————— (n) WebAuthnCredential
      (1) ————— (n) UserProgress

Course (1) ————— (n) Lesson
Course (1) ————— (n) UserProgress
Lesson (1) ————— (n) UserProgress
```

▸ Détail des modèles

Modèle	Champs clés	Description
User	id, email, username, passwordHash?, githubId?, pin?, totpSecret?, totpEnabled, trading_preference	Utilisateur principal — supporte auth multi-méthode
WebAuthnCredential	credentialId (unique, base64url), publicKey (COSE), counter, deviceType, transports	Clés FIDO2 pour biométrie/hardware key
Signal	asset, direction (BUY/SELL/HOLD), entry_price, stop_loss, take_profit, confidence (0-100), patterns (JSON), indicators (JSON)	Signal de trading généré par l'IA
Strategy	name, code (JSON StrategyRules), asset, timeframe (15m/1h/4h/1d), win_rate, profit_factor	Stratégie importée depuis PDF via Claude
Report	month, year, total_signals, buy/sell/hold_signals, win_rate, avg_confidence, total_pnl_estimate	Rapport mensuel agrégé
AuthLog	userId?, action, result, ip?, detail	Audit de tous les événements d'authentification
Course	title, description, level (DEBUTANT/INTERMEDIAIRE/AVANCE/EXPERT), category, duration, totalLessons	Cours de formation
Lesson	courseId, title, content, videoUrl?, type (VIDEO/ARTICLE/QUIZ), order	Leçon d'un cours
UserProgress	userId, courseId?, lessonId?, completed, score?	Progression d'un utilisateur dans la formation

3.4 API REST — Endpoints complets

► **Authentification (`/auth`)**

Méthode	Endpoint	Description
POST	<code>/auth/register</code>	Inscription email + mot de passe
POST	<code>/auth/login</code>	Connexion email + mot de passe
POST	<code>/auth/magic-link/request</code>	Demander un lien de connexion magique
POST	<code>/auth/magic-link/verify</code>	Vérifier le token du magic link
GET	<code>/auth/github</code>	Initier la connexion GitHub OAuth
GET	<code>/auth/github/callback</code>	Callback OAuth GitHub
POST	<code>/auth/2fa/verify</code>	Vérifier OTP email (facteur 2)
POST	<code>/auth/pin/verify</code>	Vérifier PIN utilisateur (facteur 3)
POST	<code>/auth/pin/setup</code>	Configurer le PIN (première connexion)
POST	<code>/auth/set-password</code>	Définir un mot de passe (users magic link)
POST	<code>/auth/refresh</code>	Rafraîchir l'access token via cookie httpOnly
POST	<code>/auth/logout</code>	Déconnexion + suppression tokens Redis
GET	<code>/auth/profile</code>	Récupérer le profil utilisateur authentifié

► **Signaux de trading (`/signals`)**

Méthode	Endpoint	Description
GET	/signals	Tous les signaux de l'utilisateur connecté
GET	/signals/recent	10 signaux les plus récents
GET	/signals/statistics	Stats : total, buy/sell/hold, avg confidence
POST	/signals	Créer un signal (+ notification email à tous)

▸ **Stratégies de trading** (/strategies)

Méthode	Endpoint	Description
GET	/strategies	Stratégies de l'utilisateur
POST	/strategies/import	Importer depuis PDF/TXT via Claude AI
GET	/strategies/:id/analyze	Analyser avec Claude AI
POST	/strategies/:id/backtest	Backtester une stratégie (Python)

▸ **Simulateur** (/simulator)

Méthode	Endpoint	Description
POST	/simulator/dca	Simuler une stratégie DCA (mensualités, durée, rendement)

▸ **TOTP — 2FA** (/mfa/totp)

Méthode	Endpoint	Description
GET	/mfa/totp/status	Vérifier si TOTP est activé
POST	/mfa/totp/enroll/init	Générer QR code + secret TOTP
POST	/mfa/totp/enroll/confirm	Valider avec premier code TOTP
POST	/mfa/totp/verify	Vérifier un code TOTP
POST	/mfa/totp/disable	Désactiver TOTP

► **WebAuthn — FIDO2** (/mfa/webauthn)

Méthode	Endpoint	Description
GET	/mfa/webauthn/credentials	Lister clés FIDO2 enregistrées
POST	/mfa/webauthn/register/options	Challenge enrôlement biométrique
POST	/mfa/webauthn/register/verify	Vérifier réponse enrôlement
POST	/mfa/webauthn/auth/options	Challenge authentification biométrique
POST	/mfa/webauthn/auth/verify	Vérifier réponse authentification
DELETE	/mfa/webauthn/credentials/:id	Révoquer une clé FIDO2

► **Formation** (/formation)

Méthode	Endpoint	Description
GET	/formation/courses	Liste des cours publiés (avec % progression)
GET	/formation/courses/:id	Détail cours + leçons + progression
GET	/formation/lessons/:id	Détail d'une leçon
POST	/formation/progress	Marquer une leçon comme complétée
GET	/formation/my-progress	Progression globale de l'utilisateur

► **Santé & Métriques**

Méthode	Endpoint	Description
GET	/	Health check basique (retourne 200)
GET	/health	Health check détaillé
GET	/metrics	Métriques Prometheus (prom-client format)

SECTION 4 — FONCTIONNALITÉS DÉVELOPPÉES

4.1 Authentification Multi-Facteurs

▸ Ce que ça fait pour l'utilisateur

L'utilisateur peut se connecter via 4 méthodes différentes : mot de passe classique, lien magique envoyé par email, connexion GitHub OAuth, ou PIN. En plus, il peut activer un 2e facteur via une application TOTP (Google Authenticator) ou via sa biométrie (empreinte digitale, Face ID) grâce à WebAuthn FIDO2.

▸ Implémentation technique

Flux Email + Mot de passe :

1. `POST /auth/login` → `AuthService.login()`
2. `bcrypt.compare(password, passwordHash)` avec protection brute-force (3 tentatives → verrouillage 30 min, TTL Redis)
3. Tracking par IP : alerte `SUSPICIOUS_IP` après 10 échecs en 1h
4. `issueTokens()` → `JWT access token (15 min)` + `Redis refresh token (TTL 7j)` + cookie `httpOnly`

Flux Magic Link :

1. `POST /auth/magic-link/request` → `crypto.randomBytes(32).toString('hex')`
2. Stockage Redis : `magic: = email` (TTL 15 min)
3. `EmailService.sendMagicLink()` → template HTML avec lien
4. `POST /auth/magic-link/verify` → `REDIS.get()` + `findOrCreate User` → tokens JWT

Flux TOTP :

1. `POST /mfa/totp/enroll/init` → `authenticator.generateSecret(20)` → QR code SVG via `qrcode`
2. Secret stocké temporairement Redis (TTL 10 min) en attente confirmation
3. `POST /mfa/totp/enroll/confirm` → `authenticator.verify()` → `AES-256-GCM encrypt(secret)` → `UPDATE User.totpSecret`

4. Tolérance ± 1 période (30s) pour décalages horloge

Flux WebAuthn :

1. `POST /mfa/webauthn/register/options` → `generateRegistrationOptions()` → challenge
Redis (TTL 5 min)
2. `POST /mfa/webauthn/register/verify` → `verifyRegistrationResponse()` → `CREATE WebAuthnCredential`
3. Authentification : challenge → `verifyAuthenticationResponse()` → counter++ (protection replay attack)

► Fichiers concernés

- `auth/auth.service.ts` (520 lignes)
- `auth/guards/jwt-auth.guard.ts`
- `mfa/totp/totp.service.ts` (195 lignes)
- `mfa/webauthn/webauthn.service.ts` (295 lignes)
- `pages/auth/` (10 pages)

4.2 Dashboard Temps Réel

► Ce que ça fait pour l'utilisateur

Le dashboard affiche les KPIs clés en temps réel : capital total estimé, win rate, nombre de signaux, performance. Les données se rafraîchissent automatiquement sans rechargement de page.

► Implémentation technique

- Composant `DashboardFuturiste.tsx` : 4 `StatCard` avec animations Framer Motion
- Appels parallèles via Axios : `signalsApi.getStatistics()` + `signalsApi.getRecent()`
- `useEffect` avec `setInterval` pour rafraîchissement périodique
- Zustand `useAuthStore` pour l'identité de l'utilisateur

► Fichiers concernés

- `components/common/DashboardFuturiste.tsx`
- `pages/dashboard/index.tsx`
- `api/index.ts`

4.3 Page Marchés — Données CoinGecko Live

► Ce que ça fait pour l'utilisateur

La page Marchés affiche les 20 principales cryptomonnaies avec prix en temps réel, variation 24h, volume, market cap et sparkline. Un indicateur LIVE clignote pour signaler les mises à jour.

► Implémentation technique

- **API Route Next.js** `/api/coingecko/markets` : proxy avec cache serveur TTL 60s
- Cache stale-while-revalidate : en cas de rate limit CoinGecko (HTTP 429), les données périmées sont renvoyées
- Rafraîchissement automatique côté client toutes les 90 secondes
- `X-Cache: HIT|MISS|STALE` header pour monitoring

► Fichiers concernés

- [pages/api/coingecko/markets.ts](#)
- [utils/serverCache.ts](#)
- [pages/markets/index.tsx](#)

4.4 Graphiques Chandeliers Japonais

► Ce que ça fait pour l'utilisateur

Sur la page détail d'une crypto, un graphique en chandeliers japonais interactif affiche les prix OHLC avec 5 timeframes (1H, 4H, 1J, 1S, 1M). L'utilisateur peut zoomer, scroller et survoler pour voir les valeurs exactes.

► Implémentation technique

- `lightweight-charts` de TradingView : import dynamique côté client (`await import('lightweight-charts')`)
- Création du chart avec `createChart()` : configuration couleurs dark mode (`#0d1117`), grille, crosshair
- `addCandlestickSeries()` : bougies vertes (`#10b981`) pour hausse, rouges (`#ef4444`) pour baisse
- Données OHLC depuis CoinGecko `/coins/{id}/ohlcv?days=N` : format `[timestamp, open, high, low, close]`
- Déduplication + tri chronologique des barres avant injection
- `ResizeObserver` pattern : `window.resize` → `chart.applyOptions({ width })`
- Skeleton loader animé pendant le chargement (chandeliers fictifs avec animation `pulse-dot`)

► Fichiers concernés

- [pages/markets/[id].tsx](frontend-web/pages/markets/[id].tsx) (575 lignes)
- [pages/api/coingecko/ohlcv/[id].ts](frontend-web/pages/api/coingecko/ohlcv/)

4.5 Import et Analyse de Stratégies PDF

► Ce que ça fait pour l'utilisateur

L'utilisateur peut uploader un document PDF ou texte contenant une stratégie de trading (ex: "Acheter quand le RSI passe sous 30 et que le MACD croise à la hausse"). Claude AI analyse le document et extrait automatiquement les règles en JSON structuré : conditions d'entrée, sortie, indicateurs, timeframe, risk management.

► Implémentation technique

1. Frontend : `POST /strategies/import` avec multipart FormData (fichier PDF)
2. Backend `StrategiesService.importFromFile()` : extraction texte (pdfjs ou lecture directe)
3. `AIService.analyzeStrategyDocument(text)` → Anthropic SDK `client.messages.create()`
4. Modèle : `claude-sonnet-4-6`, 4096 tokens max
5. System prompt : instruction stricte pour retourner UNIQUEMENT du JSON valide
6. `cleanJsonResponse()` : nettoyage des éventuels blocs markdown dans la réponse
7. `validateStrategyRules()` : validation TypeScript de la structure avant persistance
8. Prisma `CREATE Strategy` avec le JSON normalisé

► Fichiers concernés

- [ai/ai.service.ts](#) (280 lignes)
- [strategies/strategies.service.ts](#)
- [pages/strategies/import.tsx](#)

4.6 Module Formation (LMS)

► Ce que ça fait pour l'utilisateur

Un système de formation intégré avec des cours structurés par niveau (Débutant, Intermédiaire, Avancé, Expert). L'utilisateur voit sa progression globale en pourcentage, peut accéder aux leçons (vidéo, article, quiz) et marquer chaque leçon comme complétée.

► Implémentation technique

- `FormationService.getCourses(userId)` : récupère les cours publiés + calcule le % de progression par cours via une requête Prisma agrégée
- `markLessonComplete(userId, { lessonId, courseId })` : `upsert UserProgress` + recalcul % progression
- `getUserProgress(userId)` : agrégation globale (total leçons / leçons complétées × 100)
- Modèles Prisma avec enums `CourseLevel` et `LessonType`
- Seed automatique des données de formation (`seed-formation.ts`)

► Fichiers concernés

- [formation/formation.service.ts](#) (172 lignes)
- [pages/formation/](#) (3 pages)
- [prisma/seed-formation.ts](#)

4.7 Signaux de Trading (BUY/SELL/HOLD)

► Ce que ça fait pour l'utilisateur

L'IA génère automatiquement des signaux de trading sur les cryptomonnaies. Chaque signal inclut : direction (BUY/SELL/HOLD), prix d'entrée, stop-loss, take-profit, ratio risque/rendement, patterns détectés et score de confiance de 0 à 100%.

► Implémentation technique

- `SignalGenerator.generate_signal()` en Python : agrège patterns chandeliers + chart patterns + RSI + MACD + Bollinger Bands
- Score de confiance : `min(95, 50 + buy_signals × 10)` → plafond à 95%
- Stop-loss : `entry_price - (ATR × 2)`, Take-profit : `entry_price + (ATR × 3)`
- Ratio R/R : `(TP - entry) / (entry - SL)`
- `SignalsService.createSignal()` → Prisma INSERT + `notifyAllUsers()` (email batch BCC 50 recipients)
- `SignalCard.tsx` : affichage avec badge coloré (vert BUY, rouge SELL, gris HOLD)

► Fichiers concernés

- `ai-module/signal_generator.py` (169 lignes)
 - `signals/signals.service.ts`
 - `components/common/SignalCard.tsx`
-

4.8 Simulateur d'Investissement DCA

▸ Ce que ça fait pour l'utilisateur

Un simulateur de stratégie DCA (Dollar Cost Averaging) permet à l'utilisateur de projeter ses investissements. Il entre un capital initial, un montant mensuel, une durée et un taux de rendement annuel. Le simulateur calcule la valeur finale, les gains totaux et le ROI.

▸ Implémentation technique

- `DCASimulator.simulate_dca(initial_amount, monthly_investment, months, annual_return, volatility)` en Python
- Applique un rendement mensuel aléatoire (`np.random.normal`) centré sur `annual_return/12` avec la `volatility` comme écart-type
- Retourne les données mois par mois pour graphique de progression
- Frontend : formulaire React avec visualisation courbe Framer Motion

▸ Fichiers concernés

- `ai-module/dca_simulator.py`
- `simulator/simulator.service.ts`
- `pages/simulator/index.tsx`

4.9 Rapports Mensuels

► Ce que ça fait pour l'utilisateur

Chaque mois, un rapport automatique est généré : nombre de signaux BUY/SELL/HOLD, win rate du mois, confiance moyenne, meilleur et pire signal, patterns les plus fréquents, estimation P&L.

► Implémentation technique

- `ReportGenerator.generate_monthly_report(year, month)` en Python : filtre les signaux par mois, calcule toutes les métriques
- `ReportsService` NestJS : `upsert Report` (unique par userId+month+year)
- `PerformanceTracker` : suivi P&L par trade avec calcul `winning_trades / losing_trades`
- Frontend `reports/index.tsx` : sélecteur mois/année + tableau récapitulatif

► Fichiers concernés

- `ai-module/report_generator.py`
- `ai-module/performance_tracker.py`
- `pages/reports/index.tsx`

4.10 Module IA Python — Détection de Patterns

► Ce que ça fait pour l'utilisateur

En arrière-plan, le module Python analyse les données OHLCV en temps réel pour détecter des patterns : chandeliers japonais (Hammer, Doji, Engulfing), figures chartistes (Double Top/Bottom, Head & Shoulders), patterns harmoniques (Gartley, Bat, Butterfly, Crab) et vagues Elliott.

► Implémentation technique

- Exécution via `child_process.execAsync()` depuis NestJS : `python -c "import ..."`
- Données passées via fichiers JSON temporaires (dossier `temp/`)
- Architecture pipeline : CandlestickPatternDetector → ChartPatternDetector → HarmonicDetector → ScoringEngine → SignalGenerator
- Chaque détecteur retourne `dataclass Pattern(name, direction, confidence)`
- `ScoringEngine` agrège : 50% score patterns + 50% score indicateurs

► Fichiers concernés

- `ai-module/candlestick_patterns.py`
- `ai-module/chart_patterns.py`
- `ai-module/harmonic_patterns.py`
- `ai-module/scoring_engine.py`
- `ai/ai.service.ts`

SECTION 5 — STRATÉGIES DE TRADING IMPLÉMENTÉES

5.1 RSI Strategy (Relative Strength Index)

Champ	Détail
Concept	L'indicateur RSI mesure la force et la vitesse des mouvements de prix sur une échelle de 0 à 100. Sous 30 = survente (opportunité d'achat), au-dessus de 70 = surachat (opportunité de vente).
Indicateurs	RSI(14) — moyenne mobile des gains vs pertes sur 14 périodes
Signal BUY	RSI < 30 → survente, pression vendeuse épuisée, rebond probable
Signal SELL	RSI > 70 → surachat, pression acheteuse saturée, correction probable
Pondération	+2 signaux BUY/SELL dans le générateur de signal (double poids)

```
# Fichier : ai-module/indicators_calculator.py
def rsi(self, period: int = 14) -> pd.Series:
    delta = self.df['Close'].diff()
    gains = delta.where(delta > 0, 0).rolling(window=period).mean()
    losses = -delta.where(delta < 0, 0).rolling(window=period).mean()
    rs = gains / losses
    return 100 - (100 / (1 + rs))

# Utilisation dans signal_generator.py
rsi = indicators.rsi(14)[-1]
if rsi < 30:
    buy_signals += 2 # Surpondération zone de survente
elif rsi > 70:
    sell_signals += 2 # Surpondération zone de surachat
```


5.2 MACD Strategy (Moving Average Convergence Divergence)

Champ	Détail
Concept	Le MACD mesure la différence entre deux moyennes mobiles exponentielles (EMA12 - EMA26). La ligne de signal (EMA9 du MACD) permet de détecter les croisements bullish/bearish.
Indicateurs	EMA(12), EMA(26), Signal line EMA(9), Histogramme
Signal BUY	MACD line > 0 → momentum haussier (EMA rapide au-dessus de l'EMA lente)
Signal SELL	MACD line < 0 → momentum baissier

```
# Fichier : ai-module/indicators_calculator.py

def macd(self, fast: int = 12, slow: int = 26, signal: int = 9):
    ema_fast    = self.df['Close'].ewm(span=fast).mean()
    ema_slow    = self.df['Close'].ewm(span=slow).mean()
    macd_line   = ema_fast - ema_slow
    signal_line = macd_line.ewm(span=signal).mean()
    histogram   = macd_line - signal_line
    return macd_line, signal_line, histogram

# Utilisation dans signal_generator.py
macd_values = indicators.macd(12, 26, 9)
macd_line   = macd_values[-1]
if macd_line > 0:
    buy_signals += 1
else:
    sell_signals += 1
```

5.3 Bollinger Bands Strategy

Champ	Détail
Concept	Les bandes de Bollinger enveloppent le prix dans un canal basé sur la moyenne mobile et l'écart-type. Un prix qui touche la bande basse signale une sous-évaluation (BUY), la bande haute une surévaluation (SELL).
Indicateurs	$SMA(20) \pm 2 \times \text{écart-type}(20)$
Signal BUY	Prix < Bande basse → prix en-dessous de la "valeur normale" statistique
Signal SELL	Prix > Bande haute → prix au-dessus de la "valeur normale" statistique

```
# Fichier : ai-module/indicators_calculator.py

def bollinger_bands(self, period: int = 20, std_dev: float = 2):
    sma = self.df['Close'].rolling(window=period).mean()
    std = self.df['Close'].rolling(window=period).std()
    upper = sma + (std * std_dev)
    lower = sma - (std * std_dev)
    return upper, sma, lower

# Utilisation dans signal_generator.py

bollinger = indicators.bollinger_bands(20, 2)
bb_upper = bollinger['upper'][-1]
bb_lower = bollinger['lower'][-1]
if current_price < bb_lower:
    buy_signals += 1 # Prix sous la bande basse = opportunité achat
elif current_price > bb_upper:
    sell_signals += 1 # Prix sur la bande haute = opportunité vente
```

5.4 EMA/SMA Crossover Strategy

Champ	Détail
Concept	Le croisement de deux moyennes mobiles (courte période vs longue période) génère des signaux. Quand l'EMA courte croise l'EMA longue vers le haut ("golden cross") → signal BUY. Vers le bas ("death cross") → signal SELL.
Indicateurs	EMA(9), EMA(21) ou SMA(20), EMA(50) selon le timeframe
Signal BUY	EMA_courte > EMA_longue (golden cross)
Signal SELL	EMA_courte < EMA_longue (death cross)

```
# Fichier : ai-module/indicators_calculator.py

def moving_average(self, period: int = 20, ma_type: str = 'SMA') -> pd.Series:
    if ma_type == 'SMA':
        return self.df['Close'].rolling(window=period).mean()
    elif ma_type == 'EMA':
        return self.df['Close'].ewm(span=period).mean()
    raise ValueError("ma_type must be 'SMA' or 'EMA'")

# Utilisation croisement EMA
ema_fast = TechnicalIndicators(df).moving_average(9, 'EMA')
ema_slow = TechnicalIndicators(df).moving_average(21, 'EMA')
if ema_fast[-1] > ema_slow[-1] and ema_fast[-2] <= ema_slow[-2]:
    # Golden cross : EMA9 vient de croiser EMA21 vers le haut
    buy_signals += 2
```

5.5 Stochastique

Champ	Détail
Concept	L'oscillateur stochastique compare le prix de clôture actuel à la fourchette de prix sur une période donnée. Sensible aux zones de surachat/survente.
Indicateurs	%K(14, 3), %D(3) — lissages successifs
Signal BUY	%K < 20 et %K croise %D vers le haut
Signal SELL	%K > 80 et %K croise %D vers le bas

```
# Fichier : ai-module/indicators_calculator.py
def stochastic(self, period: int = 14, smooth_k: int = 3, smooth_d: int = 3):
    low_min = self.df['Low'].rolling(window=period).min()
    high_max = self.df['High'].rolling(window=period).max()
    k_percent = 100 * (self.df['Close'] - low_min) / (high_max - low_min)
    k = k_percent.rolling(window=smooth_k).mean()
    d = k.rolling(window=smooth_d).mean()
    return k, d
```

5.6 ATR — Average True Range (Gestion du risque)

Champ	Détail
Concept	L'ATR mesure la volatilité du marché. Il n'est pas un signal directionnel mais un outil de gestion du risque pour placer le stop-loss et le take-profit à des niveaux cohérents avec la volatilité réelle.
Utilisation	Stop-loss = entry $\pm$ ATR $\times$ 2, Take-profit = entry $\pm$ ATR $\times$ 3 (R/R ratio = 1.5)

```
# Fichier : ai-module/indicators_calculator.py
def atr(self, period: int = 14) -> pd.Series:
    tr1 = self.df['High'] - self.df['Low']
    tr2 = abs(self.df['High'] - self.df['Close'].shift())
    tr3 = abs(self.df['Low'] - self.df['Close'].shift())
    tr = pd.concat([tr1, tr2, tr3], axis=1).max(axis=1)
    return tr.rolling(window=period).mean()

# Application dans signal_generator.py
atr = TechnicalIndicators(self.df).atr(14)[-1]
# BUY signal
stop_loss = current_price - (atr * 2) # -2×ATR
take_profit = current_price + (atr * 3) # +3×ATR
risk_reward_ratio = (take_profit - entry) / (entry - stop_loss) # = 1.5
```

5.7 Patterns Harmoniques (Gartley, Bat, Butterfly, Crab)

Champ	Détail
Concept	Les patterns harmoniques utilisent des ratios de Fibonacci pour identifier des zones de retournement avec haute probabilité. Chaque pattern (XABCD) a des ratios précis entre ses vagues.
Indicateurs	Ratios Fibonacci : 0.382, 0.50, 0.618, 0.786, 1.272, 1.618
Signal BUY	Pattern bullish complété sur point D (zone de retournement potentielle haussière)
Signal SELL	Pattern bearish complété sur point D

```

# Fichier : ai-module/harmonic_patterns.py

@dataclass
class HarmonicPattern:
    name: str
    direction: str      # "BUY" | "SELL"
    confidence: float   # 0-100
    x_price: float
    a_price: float
    b_price: float
    c_price: float
    d_price: float      # Zone de retournement potentiel
    entry_level: float
    stop_loss: float
    take_profit: float

class HarmonicDetector:
    def detect_gartley(self) -> Optional[HarmonicPattern]:
        # XAB ratio : 0.618 de XA
        # ABC ratio : 0.382 - 0.886 de AB
        # BCD ratio : 1.272 - 1.618 de BC
        # XAD ratio : 0.786 de XA
        xab = abs(b - a) / abs(a - x) if abs(a - x) > 0 else 0
        xad = abs(d - a) / abs(a - x) if abs(a - x) > 0 else 0
        if 0.55 <= xab <= 0.70 and 0.70 <= xad <= 0.86:
            return HarmonicPattern(name='Gartley', direction='BUY', ...)

```

5.8 Vagues Elliott

Champ	Détail
Concept	La théorie des vagues Elliott postule que les marchés se déplacent en cycles de 5 vagues impulsives (tendance) suivies de 3 vagues correctives. Identifier la vague en cours permet d'anticiper le prochain mouvement.
Vague 1	Premier mouvement haussier (min 2% de gain)
Vague 2	Retracement de 15-65% de la vague 1
Vague 3	La plus forte → >150% de la vague 1
Vague 4	Retracement 15-50% de la vague 3 (ne chevauchant pas vague 1)
Vague 5	Dernière impulsion → >50% de la vague 1

```
# Fichier : ai-module/elliott_waves.py

def detect_waves(self):
    wave1 = self._find_wave_1() # Mouvement initial 2%+ sur 5+ bougies
    if not wave1:
        return {'waves': [], 'pattern': 'none', 'wave_count': 0}

    wave2 = self._find_wave_2(wave1) # Retracement 15-65% de wave1
    wave3 = self._find_wave_3(wave1, wave2) # >150% de wave1
    wave4 = self._find_wave_4(wave1, wave3) # Retracement 15-50% de wave3
    wave5 = self._find_wave_5(wave1, wave4) # >50% de wave1

    if all([wave1, wave2, wave3, wave4, wave5]):
        return {'pattern': 'complete_impulse', 'wave_count': 5,
                'overall_direction': 'BUY', 'waves': [...]}
```


5.9 Ichimoku Cloud

Champ	Détail
Concept	Le système Ichimoku est un indicateur complet combinant tendance, momentum, support et résistance en une seule vue. Le "nuage" (Kumo) sert de zone de support/résistance dynamique.
Composants	Tenkan Sen (9), Kijun Sen (26), Senkou Span A, Senkou Span B (52), Chikou Span
Signal BUY	Tenkan > Kijun ET prix > Kumo ET Chikou > prix passé
Signal SELL	Tenkan < Kijun ET prix < Kumo ET Chikou < prix passé

```
# Fichier : ai-module/ichimoku_indicator.py
def calculate(self):
    tenkan = (self.df['High'].rolling(9).max() + self.df['Low'].rolling(9).min()) / 2
    kijun = (self.df['High'].rolling(26).max() + self.df['Low'].rolling(26).min()) / 2
    span_a = (tenkan + kijun) / 2
    span_b = (self.df['High'].rolling(52).max() + self.df['Low'].rolling(52).min()) / 2
    chikou = self.df['Close'].shift(-26) # Décalage dans le futur

    # Signal BUY : tous les critères haussiers réunis
    if (tenkan[-1] > kijun[-1]
        and self.df['Close'][-1] > max(span_a[-1], span_b[-1])
        and chikou[-1] > self.df['Close'][-26]):
        return {'signal': 'BUY', 'confidence': 85}
```

5.10 Figures Chartistes

Pattern	Type	Signal	Description
Double Top	Bearish reversal	SELL	2 pics similaires → résistance testée 2×
Double Bottom	Bullish reversal	BUY	2 creux similaires → support tenu 2×
Triangle	Continuation	BUY/SELL	Convergence highs/lows → breakout imminent
Head & Shoulders	Bearish reversal	SELL	Tête + 2 épaules → fin de tendance haussière
Pennant	Continuation	BUY/SELL	Consolidation post-mouvement fort

```
# Fichier : ai-module/chart_patterns.py
def detect_double_top(self) -> Optional[Pattern]:
    highs = self.df['High'].values[-30:]
    peaks = [(i, h) for i, h in enumerate(highs)
              if h > highs[max(0,i-3):i].max() and h > highs[i+1:min(len(highs),i+4)].max()]
    if len(peaks) >= 2:
        peak1, peak2 = peaks[-2], peaks[-1]
        tolerance = peak1[1] * 0.02 # 2% tolérance
        if abs(peak1[1] - peak2[1]) < tolerance:
            return Pattern('Double Top', 'SELL', confidence=82)
```

SECTION 6 — EXEMPLES DE CODE CLÉS

► 1. Guard JWT — Protection des routes authentifiées

Fichier : `backend-code/src/auth/guards/jwt-auth.guard.ts`

Rôle : Protège toutes les routes nécessitant une authentification ; en mode DEV, accepte un header bypass.

```
@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') {
  canActivate(context: ExecutionContext): boolean | Observable<boolean> | Promise<boolean> {
    const request = context.switchToHttp().getRequest<Request>();

    // Mode DEV : bypass avec header x-dev-user-id
    if (process.env.NODE_ENV !== 'production') {
      const devUserId = request.headers['x-dev-user-id'];
      if (devUserId) {
        request.user = { id: devUserId as string };
        return true;
      }
    }

    // Sinon, valider le JWT normalement via Passport
    return super.canActivate(context);
  }
}
```

Explication : Ce guard NestJS étend `AuthGuard('jwt')` de Passport. Il est utilisé avec `@UseGuards(JwtAuthGuard)` sur tous les controllers nécessitant une auth. En production, il valide le Bearer token JWT. En développement, un header spécial permet de bypasser l'auth pour faciliter les tests.

▸ 2. Schéma Prisma — Modèle User (multi-auth)

Fichier : `backend-code/prisma/schema.prisma`

Rôle : Définit le modèle utilisateur supportant 4 méthodes d'authentification simultanément.

```
model User {
  id          String      @id @default(cuid())
  username    String      @unique
  email       String      @unique

  passwordHash String?    // null pour GitHub/Magic Link
  githubId     String?    @unique // null pour auth email
  pin          String?    // 3e facteur d'auth (bcrypt)
  totpSecret    String?    // chiffré AES-256-GCM
  totpEnabled   Boolean    @default(false)

  trading_preference String? @default("moderate")
  email_notifications Boolean @default(true)
  createdAt          DateTime @default(now())
  updatedAt          DateTime @updatedAt

  signals      Signal[]
  strategies   Strategy[]
  reports      Report[]
  webAuthnCredentials WebAuthnCredential[]
  progress     UserProgress[]
}
```

Explication : La conception du modèle User est pensée pour le multi-auth : `passwordHash` et `githubId` sont nullable, permettant à un même compte d'avoir les deux méthodes liées. Le `totpSecret` est stocké chiffré (AES-256-GCM) jamais en clair. Toutes les entités dépendantes utilisent `onDelete: Cascade`.

▸ 3. Appel Claude API — Analyse de stratégie

Fichier : `backend-code/src/ai/ai.service.ts`

Rôle : Appel l'API Anthropic pour analyser un document texte et extraire des règles de trading en JSON.

```

async analyzeStrategyDocument(text: string): Promise<StrategyRules> {
  const client = new Anthropic({ apiKey: this.config.get('ANTHROPIC_API_KEY') });

  const response = await client.messages.create({
    model:      'claude-sonnet-4-6',
    max_tokens: 4096,
    system:     SYSTEM_PROMPT, // Instruction JSON strict
    messages:   [{ role: 'user', content: text }],
  });

  const rawContent = (response.content[0] as { text: string }).text;
  const cleaned    = cleanJsonResponse(rawContent); // Retire les ```json...```
  const parsed     = JSON.parse(cleaned);

  if (!validateStrategyRules(parsed)) {
    throw new InternalServerErrorException(
      'Le JSON retourné par Claude ne correspond pas à la structure attendue.'
    );
  }

  return {
    ...parsed,
    entry_conditions: parsed.entry_conditions.map(String),
    confidence_score: Math.min(100, Math.max(0, Number(parsed.confidence_score))),
  };
}

```

Explication : La fonction envoie le texte du document à Claude avec un system prompt précis qui force la réponse en JSON pur. La fonction `cleanJsonResponse()` nettoie les éventuels blocs markdown. `validateStrategyRules()` vérifie TypeScript la structure avant persistance — si Claude hallucine un champ, l'erreur est catchée et retournée clairement.

► 4. Signal Generator Python — Multi-indicateurs

Fichier : `ai-module/signal_generator.py`

Rôle : Agrège patterns chandeliers, patterns chartistes, RSI, MACD et Bollinger pour générer un signal avec direction et confiance.

```

def generate_signal(self) -> TradingSignal:
    current_price = self.close[-1]
    atr           = TechnicalIndicators(self.df).atr(14)[-1]

    # Détection patterns
    patterns      = CandlestickPatternDetector(self.df).detect_all()
    chart_patterns = ChartPatternDetector(self.df).detect_all()

    # Calcul indicateurs
    rsi           = TechnicalIndicators(self.df).rsi(14)[-1]
    macd_line     = TechnicalIndicators(self.df).macd(12, 26, 9)[-1]
    bollinger     = TechnicalIndicators(self.df).bollinger_bands(20, 2)

    buy_signals = sell_signals = 0

    # Vote patterns
    for p in patterns + chart_patterns:
        if p.direction == "BUY": buy_signals += 1
        else: sell_signals += 1

    # Vote indicateurs
    if rsi < 30:         buy_signals += 2 # Survente = surpondéré
    elif rsi > 70:       sell_signals += 2
    if macd_line > 0:    buy_signals += 1
    else:               sell_signals += 1
    if current_price < bollinger['lower'][-1]: buy_signals += 1
    elif current_price > bollinger['upper'][-1]: sell_signals += 1

    # Décision finale
    if buy_signals > sell_signals:
        direction = "BUY"
        stop_loss = current_price - (atr * 2)
        take_profit = current_price + (atr * 3)
        confidence = min(95, 50 + buy_signals * 10)
    # ...

    return TradingSignal(asset=self.asset, direction=direction,

```



```
entry_price=current_price, stop_loss=stop_loss,  
take_profit=take_profit, confidence=confidence, ...)
```

Explication : Le système de "vote" (buy_signals vs sell_signals) est une approche d'ensemble learning simple : chaque indicateur vote pour BUY ou SELL. La confiance est proportionnelle au consensus. L'ATR sert à calibrer le stop-loss et take-profit à la volatilité réelle du marché.

► 5. Indicateurs Techniques Python — RSI + Bollinger

Fichier : `ai-module/indicators_calculator.py`

Rôle : Calcule les indicateurs techniques standard (RSI, MACD, Stochastic, ATR, Bollinger, EMA/SMA, Fibonacci) depuis des DataFrames pandas OHLCV.

```

class TechnicalIndicators:

    def rsi(self, period: int = 14) -> pd.Series:
        delta = self.df['Close'].diff()
        gains = delta.where(delta > 0, 0).rolling(window=period).mean()
        losses = -delta.where(delta < 0, 0).rolling(window=period).mean()
        rs = gains / losses
        return 100 - (100 / (1 + rs))

    def bollinger_bands(self, period: int = 20, std_dev: float = 2):
        sma = self.df['Close'].rolling(window=period).mean()
        std = self.df['Close'].rolling(window=period).std()
        return sma + (std * std_dev), sma, sma - (std * std_dev)

    def fibonacci_retracements(self, high: float, low: float) -> dict:
        diff = high - low
        return {
            '23.6%': high - (diff * 0.236),
            '38.2%': high - (diff * 0.382),
            '50.0%': high - (diff * 0.5),
            '61.8%': high - (diff * 0.618),
            '78.6%': high - (diff * 0.786),
        }

```

Explication : Calculs 100% NumPy/pandas — pas de dépendance externe à TA-Lib. Le RSI utilise la méthode Wilder (moyenne mobile exponentielle). Les Bollinger Bands utilisent l'écart-type glissant sur 20 périodes avec 2 déviations standard. Les ratios Fibonacci sont les niveaux classiques 23.6%, 38.2%, 50%, 61.8%, 78.6%.

► 6. Intégration CoinGecko — Proxy avec cache serveur

Fichier : `frontend-web/pages/api/coingecko/markets.ts`

Rôle : Route Next.js qui proxifie les appels CoinGecko avec un cache serveur TTL 60s et gestion du rate-limit (HTTP 429).

```
const TTL = 60_000; // 60 secondes

export default async function handler(_req: NextApiRequest, res: NextApiResponse) {
  // Cache HIT : réponse immédiate depuis la mémoire serveur
  const fresh = cache.get(KEY);
  if (fresh) {
    res.setHeader('X-Cache', 'HIT');
    return res.status(200).json(fresh);
  }

  try {
    const { data } = await axios.get('https://api.coingecko.com/api/v3/coins/markets', {
      params: { vs_currency: 'usd', order: 'market_cap_desc', per_page: 20, sparklin
      timeout: 10_000,
    });
    cache.set(KEY, data, TTL);
    res.setHeader('X-Cache', 'MISS');
    return res.status(200).json(data);
  } catch (err) {
    const status = axios.isAxiosError(err) ? (err.response?.status ?? 502) : 502;
    // Sur 429 (rate limit) : renvoyer les données périmées si disponibles
    if (status === 429) {
      const stale = cache.getStale(KEY);
      if (stale) {
        res.setHeader('X-Cache', 'STALE');
        return res.status(200).json(stale);
      }
    }
    return res.status(status).json({ error: 'Impossible de joindre CoinGecko' });
  }
}
```

Explication : Ce pattern "cache-aside avec stale fallback" évite de dépasser le rate limit de l'API CoinGecko gratuite (30 req/min). Le cache serveur (in-memory Node.js) garde les données 60 secondes. En cas de rate limit (HTTP 429), les données périmées sont renvoyées plutôt qu'une erreur — l'UX reste fluide.

► 7. Graphique Chandeliers — lightweight-charts

Fichier : `frontend-web/pages/markets/[id].tsx`

Rôle : Intègre TradingView Lightweight Charts pour afficher les données OHLC en graphique chandelier interactif côté client.

```

// Import dynamique pour éviter SSR crash (window n'existe pas côté serveur)
useEffect(() => {
  (async () => {
    const lw = await import('lightweight-charts');

    const chart = lw.createChart(containerRef.current, {
      width: containerRef.current.clientWidth,
      height: 420,
      layout: {
        background: { type: lw.ColorType.Solid, color: '#0d1117' },
        textColor: '#9ca3af',
      },
      grid: {
        vertLines: { color: '#161b22' },
        horzLines: { color: '#161b22' },
      },
    });

    const series = chart.addCandlestickSeries({
      upColor: '#10b981', // Vert émeraude – bougie haussière
      downColor: '#ef4444', // Rouge – bougie baissière
      borderUpColor: '#10b981',
      borderDownColor: '#ef4444',
      wickUpColor: '#10b981',
      wickDownColor: '#ef4444',
    });

    chartRef.current = chart;
    seriesRef.current = series;
  })();

  // Responsive : recalcul largeur lors du resize
  const onResize = () => {
    if (containerRef.current && chartRef.current)
      chartRef.current.applyOptions({ width: containerRef.current.clientWidth });
  };
  window.addEventListener('resize', onResize);

  return () => { window.removeEventListener('resize', onResize); chart.remove(); };
}, []);

```

```
// Injection des données OHLC dans le chart quand elles arrivent
useEffect(() => {
  if (!seriesRef.current || ohlcData.length === 0) return;
  seriesRef.current.setData(ohlcData);
  chartRef.current?.timeScale().fitContent();
}, [ohlcData]);
```

Explication : L'import dynamique (`await import('lightweight-charts')`) est essentiel pour Next.js : la bibliothèque accède à `window` et ne peut pas s'exécuter côté serveur (SSR). Les deux `useEffect` sont découplés : le premier initialise le chart une seule fois, le second met à jour les données à chaque changement de timeframe ou rafraîchissement.

► 8. Zustand Auth Store — Gestion d'état global côté client

Fichier : `frontend-web/context/authStore.ts`

Rôle : Store in-memory (sans localStorage) pour maintenir l'état d'authentification côté client avec Zustand.

```
import { create } from 'zustand';

interface AuthState {
  accessToken: string | null;
  user: { id: string; email: string; username: string } | null;
  isAuthenticated: boolean;
  setAuth: (token: string, user: AuthState['user']) => void;
  clearAuth: () => void;
}

export const useAuthStore = create<AuthState>((set) => ({
  accessToken: null,
  user: null,
  isAuthenticated: false,

  setAuth: (token, user) =>
    set({ accessToken: token, user, isAuthenticated: true }),

  clearAuth: () =>
    set({ accessToken: null, user: null, isAuthenticated: false }),
})));
```

Explication : Le choix de ne PAS utiliser localStorage est délibéré : les access tokens JWT ne doivent pas persister au-delà de la session pour limiter l'exposition. Seul le refresh token est persisté, mais dans un cookie httpOnly (inaccessible au JavaScript). L'interceptor Axios lit le `accessToken` du store et appelle automatiquement `POST /auth/refresh` si une requête retourne 401.

SECTION 7 — OUTILS ET MÉTHODOLOGIE

7.1 Outils de développement utilisés

Outil	Usage
VS Code	IDE principal — extensions TypeScript, Prisma, ESLint, Prettier, GitLens
Git + GitHub	Versioning — <code>git commit -m "feat: ..."</code> suivant Conventional Commits
Docker Desktop	Stack locale PostgreSQL + Redis + monitoring
Prisma Studio	Interface graphique base de données (inspection modèles, données)
Postman / Swagger	Test des endpoints API (collection Postman)
Prometheus + Grafana	Monitoring production — métriques latence, sessions, erreurs
Elasticsearch + Kibana	SIEM — analyse des logs de sécurité
Railway	Hébergement backend (NestJS)
Vercel	Hébergement frontend (Next.js) — CD automatique depuis GitHub
Claude Code (VSCode)	Assistance au développement — architecture, refactoring, debugging

7.2 Méthodologie

► Organisation du travail

- **Architecture modulaire NestJS** : chaque fonctionnalité dans son module (`auth/`, `mfa/`, `signals/`, `formation/`, etc.)

- **DTOs validés** : toutes les entrées API passent par des classes DTO avec class-validator
- **Separation of concerns** : Controller (routing) → Service (business logic) → Prisma (data)
- **Interceptors globaux** : `TransformInterceptor` enveloppe toutes les réponses en `{ data, statusCode, timestamp }`, `HttpExceptionFilter` normalise les erreurs

► Gestion de la sécurité

- Toutes les routes sensibles protégées par `@UseGuards(JwtAuthGuard)`
- Audit de toutes les actions auth via `LoggingService` (DB + Syslog)
- Brute-force protection implémentée au niveau service (pas de lib externe)
- Principe du moindre privilège : les tokens ont des expirations courtes

► Tests effectués

- **Tests unitaires** : `totp.service.spec.ts`, `webauthn.service.spec.ts`, `logging.service.spec.ts`, `metrics.service.spec.ts`
- **Tests d'intégration** : script `test_backend.sh` (curl multi-étapes)
- **Tests Python** : `test_ai_module.py` — validation de chaque détecteur de patterns
- **Tests manuels** : parcours complet utilisateur (register → login → 2FA → signal → report)

7.3 Compétences CDA démontrées

► Bloc 1 — Développer une application sécurisée

Compétence RNCP	Ce que j'ai développé dans Alvio
Écrire un code sécurisé	Bcrypt (12 rounds), AES-256-GCM, JWT avec expiration courte, cookies httpOnly
Gérer les authentifications	Système MFA complet : password + magic link + GitHub OAuth + TOTP + WebAuthn FIDO2 + PIN
Protéger contre les injections	NestJS ValidationPipe + class-validator sur tous les DTOs
Gérer les sessions	Redis avec TTL natif — refresh tokens auto-expirés, invalidation O(1)
Journaliser les événements de sécurité	AuthLog Prisma + Syslog RFC 5424 + Prometheus metrics
Protection brute-force	Compteur d'échecs Redis par user + par IP, verrouillage automatique 30 min
Tests de sécurité	Tests unitaires des services MFA, tests manuels des flux auth

► Bloc 2 — Concevoir et développer une application fullstack

Compétence RNCP	Ce que j'ai développé dans Alvio
Modéliser la base de données	Schéma Prisma : 9 modèles, relations 1:N, enums, index, contraintes unique
Développer une API REST	35+ endpoints RESTful avec NestJS, DTOs, guards, interceptors
Développer une interface web	20+ pages Next.js, composants React réutilisables, Tailwind CSS responsive
Développer une application mobile	Application React Native (mobile/)
Intégrer des services tiers	Claude API (Anthropic), CoinGecko API, SMTP (Nodemailer), GitHub OAuth
Gérer l'état côté client	Zustand store in-memory, Axios interceptors avec refresh automatique
Graphiques interactifs	lightweight-charts (TradingView) — chandeliers japonais multi-timeframe
Module IA	12 modules Python — indicateurs, patterns, scoring, signaux, DCA simulator

► Bloc 3 — Préparer le déploiement d'une application sécurisée

Compétence RNCP	Ce que j'ai développé dans Alvio
Conteneuriser l'application	docker-compose.yml — 9 services (backend, frontend, PostgreSQL, Redis, Prometheus, Grafana, Elasticsearch, Kibana, Logstash)
Configurer l'environnement	Variables d'environnement <code>.env</code> séparées par service, secrets ne commit pas
Mettre en place le monitoring	Prometheus + Grafana + prom-client (métriques custom : latence auth, sessions actives, locks)
Mettre en place le SIEM	Logstash + Elasticsearch + Kibana pour logs de sécurité via Syslog UDP RFC 5424
Déploiement continu	Vercel (frontend) + Railway (backend) — CD depuis GitHub
Documentation technique	Schéma Prisma auto-documenté, Swagger (NestJS), README

SECTION 8 — CHIFFRES ET MÉTRIQUES

8.1 Volume de code

Métrique	Valeur
Fichiers TypeScript (backend + frontend)	122 (68 backend + 54 frontend)
Fichiers Python (module IA)	15
Lignes de code approximatif	~8 500 lignes
Lignes backend TypeScript	~4 200 lignes
Lignes frontend TypeScript/TSX	~3 200 lignes
Lignes Python (IA)	~1 100 lignes
Lignes Prisma schema	216 lignes

8.2 Architecture backend

Métrique	Valeur
Modules NestJS	17 (App, Auth, MFA-TOTP, MFA-WebAuthn, AI, Signals, Strategies, Simulator, Formation, Reports, Payments, Users, Database, Redis, Email, Logging, Metrics)
Endpoints API REST	35+
Services NestJS	9 (Auth, AI, Signals, Strategies, Formation, Simulator, TOTP, WebAuthn, Logging, Metrics, Email)
Guards / Interceptors	3 (JwtAuthGuard, TransformInterceptor, HttpExceptionFilter)
DTOs validés	12

8.3 Base de données

Métrique	Valeur
Modèles Prisma	9 (User, WebAuthnCredential, Signal, Strategy, Report, AuthLog, Course, Lesson, UserProgress)
Enums Prisma	2 (CourseLevel, LessonType)
Index	8
Contraintes unique	5

8.4 Frontend

Métrique	Valeur
Pages Next.js	22 (dont 10 pages auth flow)
Composants React	13
API Routes (proxy)	4 (markets, coins/:id, ohlc/:id)
Stores Zustand	1 (authStore — in-memory)
Hooks custom	1 (useAuth)

8.5 Module IA Python

Métrique	Valeur
Modules Python	15
Algorithmes trading implémentés	12 (RSI, MACD, Bollinger, Stochastic, ATR, EMA/SMA, Fibonacci, Ichimoku, Elliott Waves, Harmonic Patterns, Candlestick Patterns, Chart Patterns)
Patterns chandeliers détectés	5 (Hammer, Doji, Engulfing, Shooting Star, 3 White Soldiers)
Figures chartistes détectées	5 (Double Top/Bottom, Triangle, H&S, Pennant)
Patterns harmoniques détectés	4 (Gartley, Bat, Butterfly, Crab)

8.6 Infrastructure

Métrique	Valeur
Services Docker	9
Métriques Prometheus	6 (auth_attempts, auth_latency, mfa_enrollments, active_sessions, http_errors_5xx, account_locks)
Templates email HTML	3 (magic link, OTP, signal notification)
Variables d'environnement	18

Résumé exécutif

Alvio est une plateforme fullstack de trading IA de niveau production démontrant la maîtrise complète du cycle de développement logiciel :

- **Architecture backend solide** : 17 modules NestJS, authentification multi-facteurs bancaire, audit de sécurité complet
- **Frontend moderne** : 22 pages Next.js avec graphiques temps réel, animations, thème adaptatif
- **Intelligence artificielle** : intégration Claude claude-sonnet-4-6 + 12 algorithmes Python de trading algorithmique
- **Infrastructure DevOps** : stack Docker complète (9 services), monitoring Prometheus/Grafana, SIEM Elasticsearch
- **Sécurité enterprise** : bcrypt, AES-256-GCM, FIDO2 WebAuthn, TOTP RFC 6238, brute-force protection, IP tracking

Ce projet démontre la capacité à concevoir, développer et déployer une application web fullstack sécurisée de niveau professionnel, couvrant l'intégralité des blocs de compétences du titre RNCP 37873 CDA niveau 6.